

Computer Netzwerke und verteilte Systeme Summer 2026



Exercise 4: Routing

Prof. Dr. Marco Zimmerling
Networked Embedded Systems Lab - NES

cnuvs@nes.informatik.tu-darmstadt.de
<https://nes-lab.org/>

Deadline: The deadline for this exercise is **19.05.2026, 23:55** and in total **28 bonus points** can be achieved.

Bonus points: *Computer Netzwerke und verteilte Systeme* features a bonus point system to acknowledge first-rate performance. Exercise points count as bonus. Throughout the entire course, a bonus of up to a maximum of 0.7 can be obtained. The bonus does not help in passing the exam: You need to pass the exam with a 4.0 or better before any bonus is applied.

Group size: The recommended group size for submissions is 3. You may also submit your exercises as a group of 2 or alone if you do not find a group.

Important: Every theoretical task and subtask require proper sources. Submissions without proper sources will automatically receive 0 points.

For most of the task the slides of the lecture will be enough. For **research** task, use the materials provided in the lecture, or the internet.

Refrain from copying and pasting your answers, as this is considered plagiarism and will have **consequences**, as will using AI tools to create your answers. Instead, explain your answers in your own words and cite your sources!

Write down your answers in a legibly and unambiguously way, either by creating formatted documents (LaTeX, Word, typst, etc.) or by handwriting (**ensure that your handwriting is legible**).

Refrain from using colors that create less contrast between the text and the background. For marking tasks you may use blue and orange.

Task 1: Group Exercise: Distance Vector Routing

Assume a network topology as shown in Figure 1. Nodes are circled and noted by an alphabetical letter and edges are the connection between the nodes with a timer. For the following subtasks, assume an existence of perfectly synchronized timers for each node, which will force all nodes to send their updates periodically and simultaneously. Given is an example distance vector table in Table 1, for all of your tables have the same sorting as the given example table. This means that nodes are sorted alphabetically from left to right under the via column and nodes are sorted alphabetically from top to bottom on the right side of the destination row.

For the initialization phase and all rounds, mark all changing values with orange and when a new minimum for the respective row after the initialization phase exists, mark them with blue. If you want to use other colors than the given ones, write out which color means which marking.

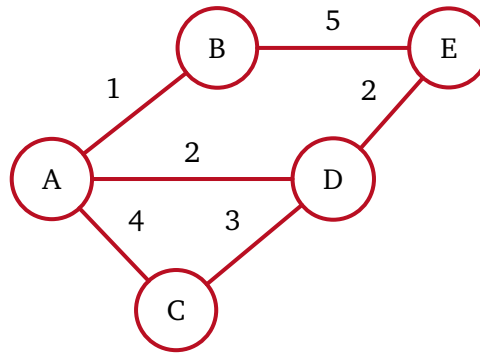


Figure 1: Topology for task 1

D^A		Via		
		B	C	D
Destination	B			
	C			
	D			
	E			

Table 1: Example layout for task 1

- Initialization phase: Calculate the distance tables for all nodes in the initialization phase, before any updates are sent to neighboring nodes from Figure 1. Do not forget to mark changing values and fill out the complete distance table for each node.
- Updates: As described, assume perfectly synchronized timers such that all nodes send their upgrades periodically and simultaneously. For each round until the stable state is reached, give the complete distance tables for all nodes and mark all values that change values and are new minima compared to the previous round.

Task 2: Group Exercise: Distance Vector Routing Link Failure

Consider the topology shown in Figure 2. The following distance tables represent the stable state **before** the link between B and C fails.

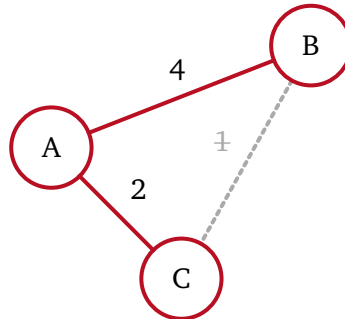


Figure 2: Topology for Task 2. The dashed edge indicates the failed link between B and C.

Stable state before the link failure:

D^A		Via	
		B	C
Dest.	B	4	3
	C	5	2

D^B		Via	
		A	C
Dest.	A	4	3
	C	6	1

D^C		Via	
		A	B
Dest.	A	2	5
	B	5	1

- Starting from the given distance tables, first **update** the entries directly affected by the failed link.
- Then **compute** the **complete distance tables** for all nodes for each synchronous update round until a **new stable state** is reached. **Mark** all values that change compared to the previous state.

Task 3: Routing (6 Points)

- Explain the difference between **routing** and **forwarding** in the context of computer networks. (1 P)
- Compare **Link State Routing** and **Distance Vector Routing**. Name 3 key differences. (1.5 P)
- How many update rounds, excluding initialization, does **DVR** require to guarantee that the shortest path between two nodes has been found? Give a reason for your answer. (1 P)

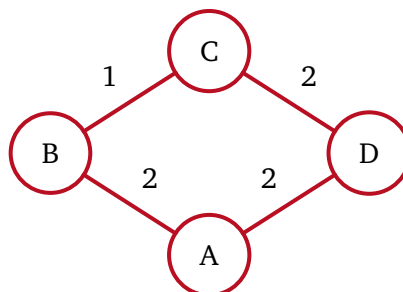


Figure 3: Topology for task 3c

- How do you know whether you have reached the last round in **DVR** without calculating the next round to see if anything changes? (1 P) (**Attention:** A reference for this subtask is not mandatory.)
- Name and Explain the problem that can occur when a **link failure** takes place. Also name the mechanism used in small networks to solve that problem. (1.5 P)

Attention: Submissions without references will not be graded! Please add sources.

Task 4: Distance Vector Routing (17 Points)

Assume a network topology as shown in Figure 4. Nodes are circled and noted by an alphabetical letter and edges are the connection between the nodes with a timer. For the following subtasks, assume an existence of perfectly synchronized timers for each node, which will force all nodes to send their updates periodically and simultaneously. Given is an example distance vector table in Table 2, for all of your tables have the same sorting as the given example table. This means that nodes are sorted alphabetically from left to right under the via column and nodes are sorted alphabetically from top to bottom on the right side of the destination row.

For the initialization phase and all rounds, mark all changing values with orange and when a new minimum for the respective row after the initialization phase exists, mark them with blue. If you want to use other colors than the given ones, write out which color means which marking.

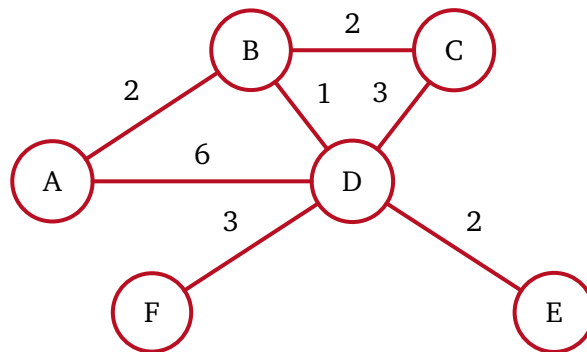


Figure 4: Network topology for Task 4 a and b

D^A		Via	
		B	D
Destination	B		
	C		
	D		
	E		
	F		

Table 2: Example distance-table layout for node A

- Initialization phase: Calculate the distance tables for all nodes in the initialization phase, before any updates are sent to neighboring nodes from Figure 4. Do not forget to mark changing values and fill out the complete distance table for each node. (3 P)
- Updates: As described, assume perfectly synchronized timers such that all nodes send their upgrades periodically and simultaneously. For each round until the stable state is reached, give the complete distance tables for all nodes and mark all values that change values and are new minima compared to the previous round. (7 P)
- Changed Link Cost: Now, assume that the link cost between A and D is 1 instead of 6 (see topology in Figure 5). Starting again with empty distance tables, repeat the distance vector algorithm from the beginning. For each round until the stable state is reached, give the complete distance tables for all nodes and mark all values that change and are new minima compared to the previous round. **Do not use the results from Task 3b as the starting point.** (7 P)

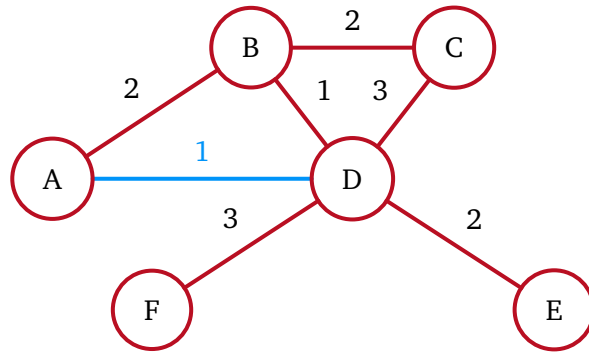


Figure 5: Network topology for Task 4 c

Task 5: Distance Vector Routing Link Failure (5 Points)

Consider the topology shown in Figure 6. The following distance tables represent the stable state **before** the link between C and D fails.

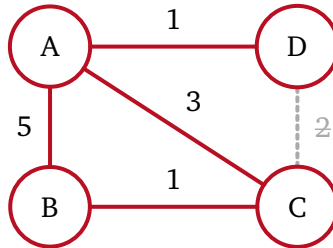


Figure 6: Topology for Task 5. The dashed edge indicates the failed link between C and D.

Stable state before the link failure:

D^A		Via		
		B	C	D
Destination	B	5	4	4
	C	6	3	3
	D	8	5	1

D^B		Via	
		A	C
Destination	A	5	4
	C	8	1
	D	6	3

D^C		Via		
		A	B	D
Destination	A	3	5	3
	B	7	1	5
	D	4	4	2

D^D		Via	
		A	C
Destination	A	1	5
	B	5	3
	C	4	2

- Starting from the given distance tables, first **update** the entries directly affected by the failed link. (2 P)
- Then **compute** the **complete distance tables** for all nodes for each synchronous update round until a **new stable state** is reached. **Mark** all values that change compared to the previous state. (3 P)